

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 8

JavaScript-Grundlagen

8.1 Allgemeines zu JavaScript

JavaScript ist eine Skriptsprache, die hauptsächlich zur Erzeugung dynamischer Inhalte im Web eingesetzt wird. Die folgenden Abschnitte geben einen allgemeinen Überblick über die Sprache.

8.1.1 Was ist JavaScript?

JavaScript ist eine interpretierte Skriptsprache, d. h. in JavaScript verfasste Scripts werden von einem Interpreter¹ abgearbeitet und entsprechend ausgeführt² (das Gegenteil sind kompilierte Sprachen, bei denen ein Compiler eine ausführbare Datei erzeugt). Die Sprache wird gerne als „Programmiersprache fürs Web“ bezeichnet, da sie hauptsächlich für Webseiten zum Einsatz kommt. Die Anweisungen werden dabei in HTML eingebettet und beeinflussen die jeweilige HTML-Seite.

Die Syntax von JavaScript entspricht in großen Teilen der von C/C++ oder Java, der größte Unterschied (neben der Tatsache, dass JavaScript interpretiert und nicht kompiliert wird) ist das schwache Typkonzept von JavaScript. Variablen haben keinen fixen Datentyp (wie z. B. in Java oder C: dort kann `int nummer` nur Integer-Werte beinhalten), sondern können verschiedene Typen aufnehmen.

JavaScript kann – wie von sehr vielen Sprachen bekannt – imperativ (dabei sowohl objektorientiert als auch prozedural) programmiert werden, besitzt aber auch funktionale³ Elemente. Die Sprache weist einen Garbage Collector auf, der den Speicher aufräumt. Dies ist analog zu Java und ein Unterschied zu C, wo das Speichermanagement selbst erfolgen muss.

8.1.2 Terminologie

JavaScript selbst kann in drei grobe Teile unterschieden werden: Core JavaScript, Client-Side JavaScript und Server-Side JavaScript.

¹<https://de.wikipedia.org/wiki/Interpreter>

²Moderne Web verwenden jedoch Just-In-Time Compilation (JIT), um wichtige oder sich wiederholende Codeteile effizienter auszuführen.

³https://de.wikipedia.org/wiki/Funktionale_Programmierung

Core JavaScript

Mit *Core JavaScript* werden alle sprachlichen Funktionalitäten der Sprache bezeichnet. Also die Art, wie Variablen definiert werden, welche Datentypen es gibt, wie Objekte verwendet werden usw. Dieser Teil von JavaScript ist offiziell von der European Computer Manufacturers Association (ECMA) standardisiert und heißt *ECMAScript*, die Nummer des Standards lautet ECMA-262 [3].

Mit Core JavaScript, d. h. ECMAScript, sind beliebige Algorithmen formulierbar. Im Gegensatz zu den meisten Programmiersprachen hat ECMAScript aber kein Input- und Output-Konzept. Dieses muss von der Umgebung, in der ECMAScript eingesetzt wird (im Regelfall ein Webbrowser), zur Verfügung gestellt werden. Dies geschieht über Client-Side JavaScript.

Client-Side JavaScript

Client-Side JavaScript ermöglicht es Entwickler*innen, auf die verschiedensten Dinge rund um eine Website zuzugreifen:

- **Inhalte der Website:** Über das so genannte Document Object Model (DOM, mehr dazu in den Vorlesungen 9 und 10) kann auf alle Elemente einer Website zugegriffen werden. Dabei können Inhalte einfach ausgelesen oder auch verändert werden.
- **Eigenschaften des Browser-Fensters:** Es können eine Vielzahl an Eigenschaften wie die Größe oder Position oder auch das Vorhandensein einzelner Bestandteile (wie z. B. einer Scrollleiste) überprüft und teilweise auch verändert werden.
- **Events:** Wann immer Benutzer*innen mit einer Website interagieren, werden dabei im Webbrowser Ereignisse (so genannte Events) ausgelöst. Zum Beispiel beim Klick auf einen Button, beim Drücken einer Taste etc. Auf diese Ereignisse kann zeitnah reagiert und somit Interaktivität erreicht werden.

Server-Side JavaScript

Schließlich existiert JavaScript auch auf der Serverseite. *Server-Side JavaScript* ist Code, der in eigenen JavaScript-Engines auf Webservern läuft und dort wie andere serverseitige Programmier- und Skriptsprachen zum Einsatz kommt. D. h. es werden damit Webseiten komplett generiert und nicht nur bestehende Teile verändert bzw. dynamisch gestaltet.

Die erste serverseitige JavaScript-Implementierung wurde bereits 1995 von Netscape realisiert, der große Durchbruch kam jedoch erst ab dem Jahr 2009 mit dem Aufkommen bzw. der Verbreitung von Node.js⁴. Node.js wird im dritten Semester in Web Architectures (WAR3VO) thematisiert.

8.1.3 Versionen, Geschichte und Browserunterstützung

JavaScript wurde im Jahr 1995 von Brendan Eich, einem Entwickler bei Netscape, entworfen und zum ersten Mal mit Netscape 2.0 im Jahr 1996 ausgeliefert. Es hieß damals noch *LiveScript* wurde aber bald im Rahmen eines – eigentlich nicht sehr klugen – Marketingschachzugs auf JavaScript umbenannt. Es sollte damals die Popularität der

⁴<https://nodejs.org/>

noch jungen Programmiersprache Java dazu verwendet werden, die eigene Skriptsprache bekannter zu machen. Dies führte aber im Endeffekt dazu, dass JavaScript bis heute oft mit Java verwechselt oder als „vereinfachte“ Version von Java dargestellt wird. Dies ist jedoch nicht korrekt, denn JavaScript und Java haben (bis auf einige syntaktische Ähnlichkeiten) nichts miteinander gemeinsam. Daher ist Folgendes wichtig:

JavaScript ist nicht Java: Es ist kein Dialekt von Java, keine einfache Version von Java, nicht Java für Webbrowser – die Sprachen sind nicht miteinander verwandt und haben nichts miteinander zu tun.

Microsoft, damals mitten im bereits in Vorlesung 1 erwähnten Browserkrieg, entwickelte darauf hin seine eigene Skriptsprache namens *JScript*, die in Internet Explorer 3 zum Einsatz kam. Darauf hin reichte Netscape seine Version zur Standardisierung bei der ECMA ein. Der erste ECMAScript-Standard wurde im Juni 1997 veröffentlicht. Im August 1998 folgte die zweite Edition. 1999 wurde schließlich ECMAScript Edition 3 fertig gestellt, die zehn Jahre lang bis zum Jahr 2009 als stabiler Standard galt. Version 4 wurde nie veröffentlicht, sodass im Dezember 2009 Edition 5 herausgebracht wurde. 2011 wurde darauf aufbauend Version 5.1 in Einklang mit dem entsprechenden ISO-Standard definiert. 2015 wurde mit Version 6 (eigentlich ECMAScript 2015) eine große Anzahl an Weiterentwicklungen eingeführt und ein jährlicher Veröffentlichungszyklus beschlossen. So erschienen seitdem ECMAScript 2016–2024 und die aktuelle Version, ECMAScript Edition 2025 [3]. Diese stammt vom Juni 2025. ECMAScript wird laufend weiterentwickelt. Die aktuelle Entwicklerversion (derzeit ECMAScript 2026) kann unter <https://github.com/tc39/ecma262> eingesehen werden, unter <https://tc39.es/ecma262/> findet sich der dazugehörige Dokumentenentwurf.

ECMAScript 5 ist laut [8] derzeit in allen aktuellen Browsern flächendeckend implementiert. Version 2015 (6.0) wird so gut wie vollständig unterstützt: führend ist Safari 26, der 100 % aller Funktionen implementiert, danach folgen Chrome 143, Edge 143, Opera 112 und Firefox 145 mit 98 %. Funktionalitäten der 2016er bis 2026er Entwürfe sind ebenso gut verfügbar: Alle Browser implementieren 99 bis 97 % davon.

8.2 Lexikalische Struktur

Die lexikalische Struktur einer Programmiersprache beschreibt die Grundregeln, die für die Sprache gelten.

8.2.1 Zeichensatz

JavaScript-Programme werden in *Unicode* geschrieben, seit ECMAScript Edition 3 kann Unicode auch an allen Stellen des Programms verwendet werden. Zuvor war es nur in Kommentaren und Strings möglich.

8.2.2 Groß- und Kleinschreibung

Groß- und Kleinschreibung spielt in JavaScript eine große Rolle d.h. die Sprache ist „case-sensitive“. Schlüsselwörter wie `while` oder `if` müssen immer klein geschrieben

werden (nicht `WHILE` oder `If`), bei Variablennamen sind `online`, `Online`, `OnLine` und `ONLINE` vier unterschiedliche Variablen.

Vorsicht ist bei HTML-Attributen gegeben: auf diese kann in JavaScript über ihre Namen zugegriffen werden. Auch wenn in HTML der Attributname `TITLE` erlaubt und mit `title` gleichzusetzen ist, muss das Attribut in JavaScript *immer* als `title` verwendet werden.

8.2.3 Strichpunkte

JavaScript-Anweisungen werden immer mit einem *Strichpunkt* (`;`) beendet. Die Sprachdefinition lässt es zu, dass Code, der in einer eigenen Zeile steht, auch ohne Strichpunkt geschrieben werden kann, davon sei aber stark abgeraten, da es eine große Fehlerquelle darstellt.

Das folgende Beispiel (hier jeweils mit einem Strichpunkt), könnte auch ohne geschrieben werden:

```
1 a = 3;  
2 b = 4;
```

Dasselbe Beispiel ohne Zeilenumbruch verlangt zwingend ein Semikolon:

```
1 a = 3; b = 4;
```

8.2.4 Kommentare

JavaScript kennt ein- und mehrzeilige *Kommentare*. Sie sind mit den Kommentaren in Java identisch. Ein einzeliges Kommentar wird als hinter zwei Schrägstriche, ein mehrzeiliges zwischen Schrägstrich und Stern sowie Stern und Schrägstrich gesetzt.

```
1 // Ein einzeliges Kommentar  
2 /* Ein  
3    mehrzeiliges  
4    Kommentar */
```

8.3 Datentypen

Als *Datentypen* werden in einer Programmiersprache alle Werte bezeichnet, die manipuliert werden können. Unterschieden wird zwischen primitiven Datentypen wie Zahlen, Literalen oder booleschen Werten sowie komplexen Datentypen wie Arrays und Objekten. Hier sollen zunächst die primitiven Datentypen und Arrays erläutert werden, Objekte folgen ausführlich und gesondert in Vorlesung 9.

8.3.1 Zahlen

JavaScript repräsentiert quasi alle *Zahlen* intern als Gleitkommazahlen nach IEEE 754 Standard [5] (dies entspricht `double`-Zahlen in Java). Dennoch können in JavaScript genauso Ganzzahlen (in den verschiedensten Zahlensystemen) angegeben werden. Der Datentyp lautet schlicht `Number`.

Ganzzahlen

Ganzzahlen können als Dezimal-, Hexadezimal-, Oktal- oder Binärzahlen angegeben werden.

- Ganzzahlen zur Basis 10 werden wie gewohnt angegeben:
 - 0,
 - -3,
 - 1234.
- Hexadezimalzahlen (Basis 16) werden durch ein vorangestelltes „0x“ angegeben:
 - 0xff,
 - 0xa5,
 - 0x145.
- Oktalzahlen (Basis 8) werden durch ein vorangestelltes „0o“ angegeben:
 - 0o377,
 - 0o27,
 - 0o11.
- Binärzahlen (Basis 2) werden durch ein vorangestelltes „0b“ deklariert:
 - 0b11111111,
 - 0b0101,
 - 0b010011010010.

Gleitkommazahlen

Gleitkommazahlen werden mit Dezimalpunkt angegeben. Danach folgen die Nachkommastellen oder optional auch eine Exponentialsyntax. Letztere wird mit einem „e“, gefolgt vom Exponenten notiert und bedeutet $\cdot 10^x$ (sprich: „Mal zehn hoch x“).

Einige Beispiele:

- 3.1415,
- -2345.67,
- 6.02e23 (entspricht $6,02 \cdot 10^{23}$),
- 1.4738e-32 (entspricht $1,4738 \cdot 10^{-32}$).

NaN

Berechnungen oder Operationen, die keinen gültigen Zahlenwert liefern, werden durch den Wert NaN (not a number – keine Zahl) repräsentiert. Dies geschieht etwa, wenn versucht wird, eine Zeichenkette mit Buchstaben in eine Zahl umzuwandeln.

BigInt

Der Standard-Datentyp für Zahlen (**Number**) stößt bei sehr großen Ganzzahlen an seine Grenzen. Da er intern auf dem IEEE 754 Standard für Gleitkommazahlen basiert, ist

der Bereich für *sichere* Ganzzahlen auf $\pm(2^{53} - 1)$ beschränkt. Zahlen, die darüber hinausgehen, verlieren ihre Präzision, was zu Berechnungsfehlern führen kann.

Um beliebig große Ganzzahlen exakt darstellen zu können, wurde der Datentyp **BigInt** eingeführt. Im Gegensatz zu **Number** wird **BigInt** nicht in einer festen Bit-Breite (64-Bit) abgelegt. Stattdessen reserviert der Datentyp dynamisch Speicher, je nachdem wie groß die gespeicherte Zahl ist. Ein **BigInt** wird erstellt, indem dem Ganzzahl-Literal ein **n** angehängt wird, z. B. `9007199254740991n`

Da sich die interne Repräsentation von **BigInt** und **Number** grundlegend unterscheidet, erlaubt JavaScript (aus Sicherheitsgründen, um Präzisionsverlust zu vermeiden) keine direkte Vermischung der beiden Typen in mathematischen Operationen. Sie müssen vor der Verrechnung explizit in den jeweils anderen Typ konvertiert werden. Zudem ist zu beachten, dass es sich um reine Ganzzahlen handelt. Eine Division schneidet Nachkommastellen ersatzlos ab.

8.3.2 Zeichenketten

Als *Zeichenkette* oder auch *String* wird eine Sequenz an Unicode-Zeichen bezeichnet, die zwischen einfachen (') oder doppelten (") Anführungszeichen gesetzt wird. Dabei ist zu beachten, dass doppelte Anführungszeichen in einem String, der mit einfachen Anführungszeichen begrenzt wird, vorkommen dürfen und umgekehrt. Zeichenketten müssen im Code immer in einer Zeile angegeben werden, es darf kein Zeilenumbruch erfolgen. Soll ein Zeilenumbruch im Text verwendet werden, so kann dieser durch `\n` erzeugt bzw. repräsentiert werden.

Einige Beispiele:

- `""` (leerer String),
- `'testing'`,
- `"3.14"` (die Zahl 3,14 als String),
- `'<p title="test">'` (doppelte Anführungszeichen sind innerhalb der einfachen möglich),
- `"Hier wird\numgebrochen"` (das Wort „umgebrochen“ steht dabei in der nächsten Zeile).

Mit dem Backslash (\) können weitere, so genannte *Escape-Zeichen* erzeugt werden:

- `\t`: Tabulator,
- `\"`: Doppelte Anführungszeichen (falls diese im Text eines in doppelten Anführungszeichen gesetzten Strings benötigt werden),
- `\'`: Einfache Anführungszeichen (falls diese im Text eines in einfachen Anführungszeichen gesetzten Strings benötigt werden),
- `\\`: Backslash.

Template Literals und Stringkonkatenation

Seit ECMAScript 2015 existiert eine erweiterte Syntax für Zeichenketten, die sogenannten *Template Literals*. Sie werden nicht von einfachen oder doppelten Anführungszeichen umschlossen, sondern vom *Backtick* (```, auch Gravis genannt).

Der größte Vorteil dieser Schreibweise ist die sogenannte *String Interpolation*. Anstatt Variablen oder Ausdrücke mühsam mit dem `+`-Operator aneinanderreihen zu müssen (Stringkonkatenation, wie etwa aus Java bekannt), können diese direkt innerhalb des Strings mittels der Syntax `${...}` eingebettet werden.

Im folgenden Beispiel werden drei Variablen angelegt (Details dazu in Abschnitt 8.4) und dann konventionell mit dem `+`-Operator aneinandergehängt. Darunter werden Template Literale verwendet. Hier können Platzhalter definiert werden, in denen die Variableninhalte interpoliert (eingesetzt) werden.

```
1 let name = "Welt";
2 let a = 5, b = 10;
3
4 // Herkömmliche Konkatenation
5 let msgAlt = "Hallo " + name + ", Ergebnis: " + (a + b);
6
7 // Modern mit Template Literals
8 let msgNeu = `Hallo ${name}, Ergebnis: ${a + b}`;
```

Ein weiteres Merkmal von Template Literals ist die Unterstützung von mehrzeiligen Strings. Ein Zeilenumbruch im Code wird dabei direkt als Zeilenumbruch im resultierenden String interpretiert, ohne dass das Steuerzeichen `\n` explizit verwendet werden muss.

Im folgenden Beispiel wird ein kleines HTML-Snippet in einer Variable `html` gespeichert. Die Zeilenumbrüche bleiben so erhalten.

```
1 let html = `

2   <h1>Titel</h1>
3   <p>Ein Absatz über mehrere Zeilen.</p>
4 </div>`;


```

8.3.3 Boolesche Werte

JavaScript kennt ebenfalls das Konzept der *booleschen Werte*, also Werte, mit denen „wahr“ oder „falsch“ dargestellt werden kann. Wie auch in anderen Sprachen üblich, werden diese zwei Zustände durch `true` und `false` repräsentiert.

8.3.4 Triviale Datentypen

JavaScript definiert darüber hinaus zwei so genannte *triviale Datentypen*, nämlich `null` und `undefined`.

`null` wird verwendet, um auszudrücken, dass eine Variable oder ein Objekt explizit keinen Wert enthält, d. h. wird eine Variable angelegt und ihr als Wert `null` zugewiesen, so wird ausgedrückt, dass noch kein konkreter Wert festgelegt wurde (aber, dass dies mit Absicht geschehen ist). `null` darf niemals mit der Zahl 0 verwechselt werden, da diese einen konkreten Wert darstellt, `null` selbst aber nicht.

`undefined` kommt dann zum Einsatz, wenn eine Variable zwar deklariert (d. h. im Code angelegt), dieser aber nie ein Wert zugewiesen wurde (auch nicht `null`). Wird der Inhalt dieser Variable dann ausgegeben, wird `undefined` angezeigt.

8.3.5 Arrays

Ein *Array* ist ein zusammengesetzter, komplexer Datentyp. Es besteht nicht nur aus einem einzelnen Wert, wie die bisher aufgelisteten Datentypen, sondern enthält mehrere davon, also entweder mehrere Zahlen, Zeichenketten oder boolesche Werte. Arrays sind Listen, d. h. mehrere Datentypen werden hintereinander angegeben und in dieser Reihenfolge gespeichert. Um Werte in einem Array abzulegen, werden diese in eckige Klammern (`[]`) gestellt und durch Beistriche voneinander getrennt.

Einige Beispiele:

- `["Yara", "Tariq", "Alex"]`,
- `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]`,
- `["WEF1VO", 2025, ["YouTube", "Microsoft Teams"]]`.

Im ersten Beispiel wird ein Array, bestehend aus drei Zeichenketten definiert, im zweiten werden zehn Ganzzahlen hinterlegt. Im dritten Beispiel zeigt sich die Besonderheit von Arrays in JavaScript. Es können durchaus auch verschiedene Datentypen in einem Array gespeichert werden. Hier ist der erste Wert eine Zeichenkette, der zweite eine Ganzzahl und der dritte ein weiteres Array, das wiederum zwei Strings enthält.

Arrays sind indexbasiert, d. h. jedem Element wird ein numerischer Index beginnend bei 0 bis $n - 1$ (wobei n die Anzahl der Elemente im Array ist) zugewiesen. Der Zugriff erfolgt über diesen Index, der hinter einer Variable (siehe dazu Abschnitt 8.4) in eckigen Klammern angegeben wird.

Dazu die wieder einige Beispiele. Es wird davon ausgegangen, dass das gemischte Array von oben (drittes Beispiel) in einer Variable `data` gespeichert ist.

- `data[0]` ergibt „WEF1VO“,
- `data[1]` ergibt 2025,
- `data[2]` ergibt `["YouTube", "Microsoft Teams"]`,
- `data[2][0]` ergibt „YouTube“ und
- `data[2][1]` ergibt „Microsoft Teams“.

Die Länge eines Arrays ist in seiner Eigenschaft `length` hinterlegt. Diese wird durch einen Punkt getrennt an den Variablennamen des Arrays gehängt. `data.length` enthält etwa die Länge des Arrays `data`. Im obigen Fall wäre dies der Wert 3, die Eigenschaft `length` zählt also nicht rekursiv (denn der dritte Eintrag im Array ist wieder ein Array, dieses wird jedoch nur als 1 gezählt).

8.4 Variablen

Variablen sind Namen, die einen Wert enthalten können. Variablen sind nötig, um in einem Programm Daten effektiv manipulieren zu können. Wie bereits in Abschnitt 8.1 erwähnt, besitzt JavaScript ein schwaches Typkonzept, d. h. es muss beim Anlegen einer Variable nicht definiert werden, welchen Inhalt sie aufnehmen soll (z. B. Ganzzahlen oder Strings). Dieses Konzept beinhaltet auch die automatische Konvertierung zwischen Datentypen. Soll etwa eine Zahl an einen String angehängt werden, wird diese zunächst automatisch in einen String umgewandelt und dann angehängt.

8.4.1 Variablennamen

Ähnlich wie XML (siehe dazu Vorlesung 7) hat auch JavaScript ein striktes *Benennungsschema*, eine vollständige und gut erläuterte Übersicht bietet dazu [2]:

- Variablennamen müssen mit einem Unicode-Buchstaben⁵, einem Underscore (_) oder einem Dollarzeichen (\$) beginnen.
- Alle folgenden Zeichen dürfen wiederum Unicode-Buchstaben und Zahlen, Underscores (_) oder Dollarzeichen (\$) sein.

Gültige Namen für Variablen sind daher z. B. folgende:

- `i`,
- `meine_variable`,
- `v13`,
- `_dummy`,
- `$str`.

Durch den Unicode-Zeichensatz sind noch weitere, in der Praxis wohl kaum verwendete Kombinationen möglich. Unter <https://mothereff.in/js-variables> existiert ein Tool, um zu überprüfen, ob ein Variablenname in JavaScript gültig ist, oder nicht.

Eine weit verbreitete Konvention bei Variablennamen ist Camel Case, d. h. Variablennamen werden mit einem Kleinbuchstaben begonnen und bei jeder Worttrennung wird ein Großbuchstabe verwendet, z. B. `noOfPersons`.

8.4.2 Deklaration

Variablen können in JavaScript explizit oder implizit *deklariert* (d. h. angelegt) werden.

Eine *explizite Deklaration* erfolgt über das Schlüsselwort `let` in Kombination mit dem gewünschten Namen:

```
1 let i;  
2 let summe;  
3 let nachricht;
```

Es können auch mehrere Variablen in einer Anweisung deklariert werden:

```
1 let i, summe, nachricht;
```

Darüber hinaus können Deklaration und Zuweisung eines Werts (Initialisierung) in einem Schritt erfolgen:

```
1 let i = 0;  
2 let summe = 3 + 5;  
3 let nachricht = "Hallo Welt!";
```

Eine *implizit deklarierte Variable* wird ohne das Schlüsselwort `let` erstellt. In diesem Fall wird einer Variable, die zuvor *nicht* mit `let` deklariert wurde, ein Wert zugewiesen. Was in anderen Programmiersprachen einen Fehler erzeugt, funktioniert in JavaScript (sofern nicht der strikter Modus – siehe Seite 8-11 – verwendet wurde). Es hat allerdings zur Folge, dass die Variable einen globalen Scope bekommt (mehr dazu in Abschnitt 8.7). Der folgende Code ist somit auch gültig:

⁵Dies sind etwa A-Z und a-z, aber auch alle unter <https://codepoints.net/search?IDS=1> aufgeführten Zeichen.

```
1 i = 3;
```

Vorsicht: Das Lesen einer noch nicht deklarierten Variable führt jedoch immer zu einem Fehler. Soll also der Wert der Variable `x` ausgelesen werden, obwohl `x` nirgends im Code bis jetzt vorgekommen ist, liefert dies einen Fehler.

Deklaration mit `var`

Das Schlüsselwort `let` wurde erst mit ECMAScript 2015 eingeführt, davor wurde `var` zur Deklaration verwendet. `let` unterscheidet sich von `var` durch den entstehenden Scope – mehr dazu in Abschnitt 8.7. Deklarationen mit `var` sehen wie folgt aus:

```
1 var i;  
2 var summe, nachricht;  
3 var nachricht = "Hallo Welt!";
```

Da ECMAScript 2015 in aktuellen Browsern vollständig unterstützt ist (siehe dazu [8]), verwendet dieses Skriptum in allen Beispielen `let` (bzw. `const` – siehe Abschnitt 8.4.4). In diversen Internet-Ressourcen oder altem Code wird jedoch noch öfters `var` zu lesen sein, da es seit Beginn die einzig gültige Syntax war.

8.4.3 Loses Typkonzept

Aufgrund des *losen Typkonzepts* ist es möglich, einer Variable verschiedene Datentypen zuzuweisen. Folgender Code funktioniert somit ohne Probleme:

```
1 let x = 42;  
2 ...  
3 x = "fourtytwo";
```

Während dies möglich und erlaubt ist, ist dennoch Vorsicht bei der Verwendung solcher Konstrukte geboten. Nicht selten führt dies zu unerwünschtem Verhalten im eigenen Script und sollte daher wirklich nur dann zum Einsatz kommen, wenn es unbedingt nötig oder beabsichtigt ist. Kommentare im Code, die erklären, warum der Datentyp geändert wird, sind dann sicherlich auch sinnvoll.

8.4.4 Konstanten

Der Wert von Variablen kann zu beliebigen Zeiten geändert werden, sogar der dahinterliegende Datentyp ist nicht festgelegt. Dennoch gibt es Fälle, in denen ein Überschreiben des Werts nicht sinnvoll oder gewünscht ist, für diesen Fall gibt es *Konstanten*. Sie können nur einmal mit einem Wert initialisiert werden und behalten diesen immer bei. Eine Konstante wird mit dem Schlüsselwort `const` deklariert.

Namen von Konstanten werden üblicherweise komplett in Großbuchstaben geschrieben, um sie deutlich von Variablen abzugrenzen. Um Wortgrenzen in Konstanten anzudeuten, wird der Unterstrich (`_`) verwendet. Im folgenden werden drei Konstanten definiert:

```
1 const LIMIT = 3600;  
2 const DEFAULT_NAME = "John Doe";  
3 const ALTER = 7 + 12;
```

Das oben gesagte gilt für primitive Datentypen wie Zahlen oder Strings. Diese sind per Definition unveränderbar, engl. *immutable*. Der Zahlenwert hinter `LIMIT` kann sich nur ändern, wenn eine neue Zahl zugewiesen wird, dabei wird aber eine neue Zahl erzeugt (nicht die bestehende verändert).

Komplexe Datentypen wie etwa Arrays (oder auch Objekte bzw. Funktionen) können ihren Wert aber ändern, ohne dass eine neue Zuweisung an die Variable erfolgt, daher gilt es zu beachten, dass `const` nur eine neue Zuweisung verhindert, die Daten dahinter sich jedoch durchaus ändern können. Der folgende Code etwa erzeugt keinen Fehler.

```
1 const myArray = [1, 2, 3];  
2 console.log("myArray vorher:", myArray); // 1,2,3  
3 myArray.push(4);  
4 console.log("myArray nachher:", myArray); // 1,2,3,4
```

An die `const`-Variable `myArray` wird ein Array mit den Inhalten 1, 2, 3 zugewiesen. Mit der Funktion `push()` (mehr über Funktionen in Abschnitt 8.7) kann in das Array am Ende ein neuer Wert (hier: 4) hinzugefügt werden. Die Zuweisung an `myArray` wird also nicht verändert (die `const`-Bedingung also nicht verletzt), der Wert des Arrays ist jetzt jedoch anders als zum Zeitpunkt der Zuweisung. In solchen Fällen ist es auch oft üblich, die normale camelCase-Schreibweise zu verwenden, um genau diese Tatsache zu verdeutlichen.

Es ist guter Stil, wann immer möglich, `const` (anstatt `let`) zur Variablendeklaration zu verwenden, um zu verdeutlichen, dass keine neue Zuweisung im Code vorgenommen werden wird. Nur wenn tatsächlich neue Zuweisungen nötig sind (etwa bei Schleifen), ist `let` vorzuziehen.

Ausgabe in Beispielen

In einigen Beispielen werden `console.log("...")`-Aufrufe verwendet. Diese ermöglichen es, eine Nachricht in die Konsole der Development-Tools der Webbrowser zu schreiben. Um sie zu sehen, müssen die Development-Tools geöffnet und dort auf den „Console“-Reiter gewechselt werden. Wird die Seite danach aufgerufen, erscheint die Nachricht. Das schreiben von Inhalten ins Dokument (d. h. die Webseite) wird in Vorlesung 10 behandelt.

8.4.5 Strikter Modus

Seit ECMAScript 5 besitzt die Sprache einen sogenannten *strikten Modus*. Dieser muss von Entwickler*innen bewusst in den eigenen Skripten aktiviert werden und erzwingt eine strengere Auslegung der JavaScript-Regeln. Was sich zunächst eher negativ anhört, hilft jedoch, sauberen Code zu erstellen und Fehler zu vermeiden, da der strikte Modus bestimmte Konstrukte, die im Standardmodus erlaubt sind, aber unter Umständen problematisch sein können, verbietet und einen Fehler anzeigt. So führen etwa implizite Variablendeklarationen zu einem Fehler, auch dürfen im strikten Modus keine Oktalzahlen mehr als 01234 definiert werden, sondern es muss die Syntax 0o1234 verwendet werden (da sie sonst zu leicht mit Dezimalzahlen verwechselt werden können). Einen guten Überblick über die vollen Auswirkungen des strikten Modus bietet MDN⁶.

⁶https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode

Um ein Script in den strikten Modus zu versetzen, muss ganz zu Beginn die folgende Angabe gesetzt werden:

```
1 "use strict";
```

In diesem Skriptum werden alle Beispiele im strikten Modus getestet, wenngleich auch die Angabe aus Platzgründen in der Regel weggelassen wird.

8.5 Operatoren

Operatoren dienen in einer Programmiersprache zur Zuweisung oder zum Vergleichen von Werten, hängen Strings aneinander oder führen bitweise Operationen durch. Die Operatoren in JavaScript gleichen sehr stark denen von C/C++ oder Java.

8.5.1 Zuweisungsoperatoren

Wie bereits in Abschnitt 8.4 verwendet, dient `=` zur *Zuweisung* von Werten an Variablen. Zum Beispiel:

```
1 i = 0;
```

Allgemeiner gesprochen erwartet der Operator auf seiner linken Seite eine Variable und auf seiner rechten einen Wert, der dieser zugewiesen wird. Der Operator ist rechtsassoziativ, d. h. mehrere Operatoren werden immer von rechts nach links ausgewertet. Ein Beispiel:

```
1 i = j = k = 0;
```

Hier wird zunächst `k` der Wert 0 zugewiesen. Danach wird `j` der Wert von `k` (auch 0) zugewiesen. Schließlich erhält `i` den Wert von `j` (wieder 0).

Zuweisung mit Operation

Zusätzlich zur normalen Zuweisung existieren auch Operatoren, die eine Zuweisung mit einer integrierten Operation durchführen. Diese sind:

- `+=`: Addition und Zuweisung,
- `-=`: Subtraktion und Zuweisung,
- `*=`: Multiplikation und Zuweisung,
- `/=`: Division und Zuweisung,
- `%=`: Modulo und Zuweisung.

Diese Operatoren können immer dann zum Einsatz kommen, wenn zum aktuellen Wert einer Variable ein neuer Wert addiert/subtrahiert/etc. wird und das Ergebnis wiederum der Variable zugewiesen werden soll.

Anstatt also

```
1 summe = summe + zusatz;
```

zu schreiben, können diese Addition und anschließende Zuweisung auch mit dem entsprechenden Operator gelöst werden:

```
1 summe += zusatz;
```

8.5.2 Vergleichsoperatoren

Ein *Vergleichsoperator* vergleicht zwei Werte und gibt immer einen booleschen Ausdruck (**true** oder **false**) zurück. Je nachdem, ob der Vergleich zutrifft, oder nicht.

Um auf Wertegleichheit zu überprüfen, dient der Operator **==**. Um zu überprüfen, ob zwei Werte ungleich sind, kommt **!=** zum Einsatz. Zwei Beispiele:

```
1 3 == 4 // false
2 "hallo" != "welt" // true
```

Um nicht nur auf (un-)gleiche Werte, sondern auch zusätzlich auf den (un-)gleichen, dahinter liegenden Typ zu überprüfen, kommen die Operatoren **===** bzw. **!==** zum Einsatz. Dies ist aufgrund des losen Typkonzepts oft sinnvoll, denn JavaScript führt – etwa bei Vergleichsoperationen – implizite Typumwandlungen durch. Werden etwa eine Zahl und ein String verglichen, so wird versucht, den String in eine Zahl umzuwandeln. Sind in der Zeichenkette dann auch nur Zahlen enthalten, gelingt dies und eine normale Vergleichsoperation ergibt **true**:

```
1 1 == "1" // ergibt true, da der String implizit umgewandelt wird
```

Mit dem Wert- und Typ-Vergleichsoperator wird auf die ursprünglichen Datentypen Rücksicht genommen, sodass der Vergleich **false** ergibt:

```
1 1 === "1" // ergibt false, da 1 eine Zahl, "1" hingegen ein String ist
```

Um auf Verhältnisse zwischen zwei Werten zu überprüfen, dienen die folgenden Operatoren:

- **<**: kleiner als,
- **>**: größer als,
- **<=**: kleiner oder gleich als,
- **>=**: größer oder gleich als.

Dazu jeweils ein Beispiel:

```
1 3 < 4 // true
2 "hallo" > "welt" // false
3 5 <= 5 // true
4 "hallo" >= "welt" // false
```

Bei Strings wird lexikografisch verglichen. In beiden Fällen kommt *h* vor *w* (d.h. $h < w$), weswegen der Vergleich immer **false** ergibt.

8.5.3 Arithmetische Operatoren

Arithmetische Operatoren repräsentieren die Grundrechnungsarten und erfüllen auch einige andere Aufgaben. Folgende stehen dabei zur Verfügung:

- **+**: Addition, Zusammenhängen (Konkatenation) von Strings,
- **-**: Subtraktion,
- *****: Multiplikation,
- **/**: Division,
- **%**: Modulo (Rest der Division),
- ******: Potenzierung (x^y),

- `++`: Inkrement,
- `--`: Dekrement.

Modulo gibt (im Unterschied zu einer Division) nicht das Ergebnis der Division, sondern den Rest einer (ganzzahligen) Division aus. Während $5 / 2$ (also $\frac{5}{2}$) 2,5 ergibt, so ist das Ergebnis von $5 \% 2$ der Wert 1, denn „5 dividiert durch 2 ist 2 mit 1 Rest“.

Die Inkrement- und Dekrement-Operatoren funktionieren ähnlich wie die Zuweisungsoperatoren mit Operation. Zu einem Wert wird 1 addiert oder subtrahiert und das Ergebnis gleich zugewiesen. Somit entsprechen

```
1 i++;
2 j--;
```

den folgenden Zuweisungen:

```
1 i = i + 1;
2 j = j - 1;
```

Ebenso sind die Operatoren (wie in C/C++), auch als Prefix-Inkrement und Prefix-Dekrement vorhanden:

```
1 ++i;
2 --i;
```

Der Unterschied ist dabei nur dann sichtbar, wenn die Operatoren in Kombination mit anderen Operationen verwendet werden:

```
1 let i = 10, j = 20;
2 let m = i-- + j; // 30
```

Hier erhält m den Wert 30, i ist danach 9 und j bleibt 20. Beim Postfix-Operator wird also zuerst i verwendet (mit dem Wert 10) und dann dekrementiert.

```
1 let i = 10, j = 20;
2 let m = --i + j; // 29
```

Im Vergleich dazu ist das Ergebnis von m hier 29, i ist wiederum 9 und j bleibt 20. Beim Prefix Operator wird also zunächst i dekrementiert und dann erst in der Operation verwendet.

8.5.4 Logische Operatoren

Logische Operatoren werden für boolesche Algebra verwendet d.h. es werden mehrere boolesche Ausdrücke miteinander verkettet und dadurch das gesamte Ergebnis (wiederum **true** oder **false**) ausgewertet. Sie kommen in der Regel zusammen mit Vergleichen und den Schlüsselwörtern **if**, **while** oder **for** vor. Folgende Operatoren gehören zu dieser Gruppe:

- `&&`: logisches UND,
- `||`: logisches ODER,
- `!`: logisches NICHT.

Das logische UND (`&&`) verknüpft zwei boolesche Ausdrücke A und B miteinander. Das Ergebnis ist dann **true**, wenn sowohl A als auch B **true** ergeben.

Mit dem logischen ODER (`||`) werden ebenfalls zwei boolesche Ausdrücke A und B miteinander verknüpft. Hier ist das Ergebnis dann **true**, wenn entweder A oder B oder beide **true** ergeben.

Das logische NICHT (!) ist ein unärer Operator (erwartet also nicht links und rechts einen Ausdruck), der vor einen Ausdruck *A* gesetzt wird und dessen booleschen Wert umkehrt. Ist *A* zunächst **true** ergibt **!A false** und umgekehrt.

Zum Beispiel:

```
1 true && false // false
2 (3 > 4) || (5 > 3) // true
3 !(3 == 3) // false
```

8.5.5 Bitweise Operatoren

JavaScript unterstützt auch die *Bit-Operatoren* & (bitweises UND), | (bitweises ODER), ^ (bitweises Exklusiv-ODER), ~ (bitweises NICHT) sowie << (bitweise Verschiebung nach links) und >> (bitweise Verschiebung nach rechts). Mehr zu Bit-Operatoren folgt in den fortführenden Programmierlehrveranstaltungen bzw. in der Lehrveranstaltung Data Analysis.

8.5.6 Sonstige Operatoren

JavaScript stellt noch eine Reihe weiterer Operatoren zur Verfügung, von denen vor allem der konditionale Operator **?:** erwähnenswert ist. Er ist die Kurzschreibweise einer **if-else**-Bedingung und wird folgendermaßen verwendet: Vor dem **?** steht ein boolescher Ausdruck, der ausgewertet wird. Ergibt er **true**, so wird der Teil zwischen **?** und **:** verwendet, bei **false** kommt der Teil nach dem **:** zum Zug. Ein konkretes Beispiel:

```
1 x > 0 ? x * y : -x * y;
```

Vor dem **?** wird zunächst der Ausdruck **x > 0** ausgewertet. Ergibt er **true** (weil **x** z.B. 5 ist), so wird **x * y** ausgeführt. Ergibt der Ausdruck **false** (weil der Wert von **x** z.B. -3 ist), so wird **-x * y** ausgeführt.

Dies könnte mit **if-else** folgendermaßen formuliert werden:

```
1 if (x > 0) {
2   x * y;
3 } else {
4   -x * y;
5 }
```

Mehr zu Kontrollstrukturen wie **if-else** im folgenden Abschnitt.

8.6 Kontrollstrukturen

Mit *Kontrollstrukturen* können Programmabläufe gesteuert werden. Es werden Verzweigungen und Schleifen unterschieden.

8.6.1 Verzweigungen

Wie in C/C++, Java und anderen Sprachen gibt es auch in JavaScript die **if-else**-Bedingung und das **switch**-Statement, um *Verzweigungen* zu realisieren.

if-Bedingungen unterliegen der folgenden Syntax:


```
1 if (Ausdruck) {  
2   Anweisung  
3 }
```

Ergibt der boolesche Ausdruck in der Klammer **true**, wird die Anweisung in den geschweiften Klammern ausgeführt. Optional kann noch ein **else**-Zweig hinzugefügt werden, der dann ausgeführt wird, wenn der boolesche Ausdruck nicht zutrifft. Folgendes Beispiel enthält **if** mit zugehörigem **else**-Zweig:

```
1 if (name !== "") {  
2   username = name;  
3 } else {  
4   username = "John Doe";  
5 }
```

Sollen mehrere Bedingungen hintereinander getestet werden, kann im **else**-Zweig wiederum eine **if**-Bedingung angereicht werden:

```
1 if (n === 1) {  
2   // Code 1...  
3 } else if (n === 2) {  
4   // Code 2...  
5 } else if (n === 3) {  
6   // Code 3...  
7 } else {  
8   // Restlicher Code, wenn nichts zutrifft.  
9 }
```

Diese Art von **if**-Kaskade ist jedoch weder einfach zu warten noch leicht zu lesen, daher kann stattdessen eine **switch**-Anweisung zum Einsatz kommen. Die generelle Syntax lautet:

```
1 switch(Ausdruck) {  
2   case Fall1:  
3     Anweisung  
4     break;  
5   case Fall2:  
6     Anweisung;  
7     break;  
8   default:  
9     Anweisung;  
10    break;  
11 }
```

Der Ausdruck in der Klammer nach dem **switch**-Schlüsselwort wird ausgewertet, danach wird überprüft ob einer der Werte in den einzelnen **case**-Ausdrücken damit übereinstimmt. Wenn ja, wird der jeweilige Code ausgeführt. Ist dies nicht der Fall, kommt der Code beim **default**-Schlüsselwort zum Zug. Hinter jedem **case**-Ausdruck muss eine **break**-Anweisung stehen, damit wirklich nur diese eine Anweisung ausgeführt wird. Fehlt das **break**-Schlüsselwort, wird der nachfolgende **case**-Abschnitt auch ausgeführt.

Das folgende **switch**-Statement gleicht der **if**-Kaskade von oben:

```
1 switch(n) {  
2   case 1:  
3     // Code 1...  
4     break;
```

```
5   case 2:
6     // Code 2...
7     break;
8   case 3:
9     // Code 3...
10    break;
11  default:
12    // Restlicher Code, wenn nichts zutrifft.
13    break;
14 }
```

8.6.2 Schleifen

JavaScript bietet wie viele andere Programmiersprachen auch die drei bekannten (klassischen) *Schleifen* **while**, **do/while** und **for**.

Eine **while**-Schleife hat die folgende Syntax:

```
1 while (Ausdruck) {
2   Anweisung
3 }
```

Solange der Ausdruck in der Klammer der **while**-Schleife **true** ergibt, wird die Anweisung im Schleifenkörper ausgeführt. Die folgende Schleife wird genau 10 Mal durchlaufen, danach ist die Bedingung nicht mehr gültig und die Schleife wird nicht mehr betreten:

```
1 let count = 0;
2 while (count < 10) {
3   console.log(count);
4   count++;
5 }
```

Während bei einer **while**-Schleife der Code erst ausgeführt wird, wenn die Bedingung als gültig anerkannt wird, wird bei einer **do/while** Schleife der Code zunächst ein Mal ausgeführt und dann überprüft, ob er wiederholt werden soll. Die Syntax sieht wie folgt aus:

```
1 do {
2   Anweisung
3 } while (Ausdruck);
```

Bei der **while**-Schleife ist ersichtlich, dass die Zählervariable, die in der Bedingung ausgewertet wird, selbst hochgezählt werden muss (durch die **count++**-Anweisung). Wird etwa auf diese Zeile vergessen, so wird eine Endlosschleife erzeugt, da die Bedingung immer **true** ergibt. Eine **for**-Schleife ist in solchen Fällen oft praktischer, da die Zählervariable bereits im Schleifenkopf enthalten ist. Eine solche Schleife besteht aus Initialisierung, Abtestung und Erhöhung der Zählervariable:

```
1 for (Initialisierung; Test; Erhöhung) {
2   Anweisung
3 }
```

Um die **while**-Schleife, die genau zehn Mal durchlaufen wurde, als **for**-Schleife zu konstruieren, ist folgender Code nötig:

```
1 for (let count = 0; count < 10; count++) {  
2   console.log(count);  
3 }
```

Alle Werte eines Arrays ausgeben: `for...of`-Schleife

Zusätzlich zu den drei klassischen Schleifentypen existiert die `for...of`-Schleife. Sie iteriert über alle Einträge eines Arrays, ohne dabei einen Zähler vorrücken zu müssen. Die prinzipielle Syntax lautet wie folgt:

```
1 for (Element in Array) {  
2   Anweisung  
3 }
```

Zunächst wird eine Variable definiert, in der bei jedem Schleifendurchlauf der aktuelle Werte des Arrays enthalten sein soll. Mit diesem Wert kann dann im Rumpf der Schleife gearbeitet werden. Folgendes Beispiel durchläuft ein einfaches Array und gibt jeden Eintrag in der Konsole aus.

```
1 let namen = ["Yara", "Tariq", "Alex"];  
2  
3 for (const element of namen) {  
4   console.log(element);  
5 }
```

8.7 Funktionen

Eine *Funktion* ist ein Block mit JavaScript-Code, der einmal definiert und beliebig oft aufgerufen werden kann. Funktionen können so genannte Parameter besitzen. Dies sind Platzhalter, die beim Funktionsaufruf mit Werten (den sogenannten Argumenten) befüllt werden und die das Berechnungsergebnis der Funktion beeinflussen. Dieses Ergebnis kann als Rückgabewert zurückgegeben werden. Dieser Rückgabewert wird dann dem Wert der Variable zugewiesen, der zuvor die Funktion zugewiesen wurde.

8.7.1 Definieren von Funktionen

Eine Funktion wird mit dem Schlüsselwort `function` *definiert*. Danach folgen der Name und ggf. durch Kommas getrennte Parameter in Klammern. Der Code, der beim Funktionsaufruf ausgeführt werden soll, steht in geschweiften Klammern. Die prinzipielle Syntax sieht wie folgt aus:

```
1 function name(param1, param2, ...) {  
2   Anweisung  
3 }
```

Die Konventionen für Funktionsnamen sind dieselben, wie für JavaScript-Variablen (siehe Abschnitt 8.4).

Die folgende Funktion gibt „Hallo Welt“ in der Konsole der Browser-Development-Tools aus.

```
1 function sayHello() {  
2   console.log("Hallo Welt!");  
3 }
```

Parameter angeben

Funktionen können Parameter enthalten, diese werden mit Komma getrennt in Klammern angegeben. Die Parameter können dann im Körper der Funktion verwendet werden. Das Beispiel zeigt die vorige Funktion mit einem Parameter, der die Nachricht flexibel übergeben lässt:

```
1 function saySomething(message) {  
2   console.log(message);  
3 }
```

Parameter, deren Werte primitive Datentypen sind (Zahlen, Zeichenketten, boolesche und triviale Datentypen) werden immer nach dem *call-by-value* Prinzip übergeben. Dies bedeutet, dass der Wert in der Funktion nur eine Kopie des angegebenen Werts ist. Wird die Variable `message` in der Funktion verändert, wirkt sich das auf die Variable außerhalb, die an die Funktion übergeben wird, nicht aus. Ein Beispiel:

```
1 // Eine Nachricht wird in text hinterlegt  
2 const text = "Der Text bleibt."  
3  
4 // Die Funktion hängt " So schauts aus." an die Nachricht dran.  
5 function saySomethingWithChange(message) {  
6   message += " So schauts aus.";  
7   console.log(message);  
8 }  
9  
10 // Die Funktion wird aufgerufen.  
11 // Ausgegeben wird "Der Text bleibt. So schauts aus."  
12 saySomethingWithChange(text);  
13 // In text steht aber immer noch "Der Text bleibt."  
14 console.log(text);
```

Objekte und auch Arrays werden jedoch *call-by-reference* übergeben. Werden derartige Parameter innerhalb der Funktion verändert, wirkt sich das auf das Objekt außerhalb auch aus. Details dazu in Vorlesung 9.

Rückgabewerte definieren

Ob die Funktion einen Rückgabewert hat, ist aus der Funktionsdefinition nicht erkennbar. Dazu muss lediglich in der letzten Zeile das Schlüsselwort **return** gefolgt vom zurückzugebenden Wert angegeben werden. Die folgende Funktion quadriert einen Wert und gibt das Ergebnis zurück:

```
1 function quadrat(x) {  
2   return x * x;  
3 }
```

8.7.2 Aufrufen von Funktionen

Wie schon zuvor bei der Parameterdefinition angedeutet, wird eine Funktion einfach durch Angabe ihres Namens mit eventuellen Argumenten in Klammern *aufgerufen*. Hat eine Funktion keine Parameter, werden die Klammern ohne Inhalt angegeben. Hat die Funktion einen Rückgabewert (**return** in der letzten Zeile), kann das Ergebnis auch gleich einer Variable zugewiesen werden. Dazu einige Beispiele:

```
1 quadrat(5); // Ruft quadrat() mit dem Wert 5 auf
2 saySomethingWithChange(message); // Ruft saySomethingWithChange() mit message als
   Argument auf
3 irgendwas(); // Ruft irgendwas() ohne Argument auf
4 const total = addValues(5, 2); // Aufruf von addValues() mit Zuweisung
```

Parameter vs. Argument

Unter *Parameter* versteht man die Angaben, die bei der Definition der Funktion in Klammern angegeben werden, also etwa *x*, wenn die Funktion als **function quadrat(x)** definiert wird.

Als *Argument* wird der Wert bezeichnet, der beim Aufruf für diesen Parameter eingesetzt wird. Bei **quadrat(5)** ist etwa 5 das Argument des Aufrufs.

Funktionen mit weniger Argumenten als Parametern aufrufen

JavaScript-Funktionen können auch mit *weniger* Argumenten aufgerufen werden, als in der Funktionsdefinition Parameter angegeben wurden. Jeder Parameter, der nicht durch ein Argument belegt wird, erhält in der Funktion dann den Wert **undefined**.

Angenommen, es gibt die folgende Funktion, die Vorname und Nachname einer Person auf die Konsole ausgibt:

```
1 function logData(first, last) {
2   console.log("Anzahl der Argumente:", arguments.length);
3   console.log("Vorname:", first);
4   console.log("Nachname:", last);
5 }
```

Mit dem Aufruf **logData("John", "Doe")** ist die Ausgabe wie folgt:

- Anzahl der Argumente: 2
- Vorname: John
- Nachname: Doe

Wird die Funktion nur mit einem Argument aufgerufen – **logData("John")** – ist die Ausgabe wie folgt:

- Anzahl der Argumente: 1
- Vorname: John
- Nachname: undefined

Funktionen mit mehr Argumenten als Parametern aufrufen

Auch der umgekehrte Fall ist möglich. Werden beim Aufruf einer Funktion mehr Argumente angegeben, als Parameter in der Signatur vorhanden sind, so werden diese von links nach rechts den Parametern zugeordnet. Alle „überschüssigen“ Argumente sind nicht über Parameter zugreifbar.

Sie sind dennoch nicht verloren, sondern können über das Array **arguments** erreicht werden. **arguments[0]** enthält das erste Argument, **arguments[1]** das zweite usw. Hat eine Funktion nun zwei Parameter, so kann überprüft werden, ob **arguments** länger als 2 ist und diese überschüssigen Werte verwendet werden.

Die Funktion **logData()** könnte wie folgt mit überschüssigen Argumenten umgehen:

```
1 function logData(first, last) {
2   console.log("Anzahl der Argumente:", arguments.length);
3   console.log("Vorname:", first);
4   console.log("Nachname:", last);
5   if (arguments.length > 2) {
6     for (let i = 2; i < arguments.length; i++) {
7       console.log("Zusätzlich:", arguments[i]);
8     }
9   }
10 }
```

Sind mehr als zwei Argumente vorhanden, wird eine Schleife über die Einträge im `arguments`-Array (beginnend beim Eintrag 2) gestartet und jeder Eintrag wird separat auf die Konsole ausgegeben.

Der Aufruf `logData("John", "Doe", 23, "Doeville")` erzeugt diese Ausgabe:

- Anzahl der Argumente: 4
- Vorname: John
- Nachname: Doe
- Zusätzlich: 23
- Zusätzlich: Doeville

Funktionsaufrufe mit einer variablen Anzahl an Argumenten

Eine flexiblere, seit ECMAScript 2015 verfügbare Variante, mit einer variablen Anzahl von Argumenten umzugehen, sind so genannte *Rest-Parameter*. Sie werden mit drei Punkten vor dem Parameternamen angegeben, z.B. `...numbers`. Eine Funktion `addValues()`, die eine beliebige Anzahl von Zahlen addieren soll, könnte mit Hilfe eines Rest-Parameters wie folgt geschrieben werden:

```
1 function addValues(...numbers) {
2   let sum = 0;
3   for (let number of numbers) {
4     sum += number;
5   }
6   return sum;
7 }
8
9 console.log("3 Argumente:", addValues(1, 2, 3)); // 6
10 console.log("5 Argumente:", addValues(1, 2, 3, 7, 11)); // 24
11 console.log("10 Argumente:", addValues(1, 2, 3, 7, 11, 81, 19, 2, 33, 10)); // 169
```

Der Aufruf kann nun mit einer beliebigen Anzahl an Argumenten, getrennt durch Beistriche erfolgen.

Der Unterschied zum `arguments`-Array ist, dass in diesem *alle* Argumente enthalten sind (auch solche, für die ein Parameter existiert), während in Rest-Parametern nur jene, ohne expliziten Namen enthalten sind (im Beispiel sind dies alle, da der Rest-Parameter der einzige ist).

Standardwerte für Parameter definieren und aufrufen

Wie ersichtlich ist, müssen nicht alle Parameter beim Aufruf einer Funktion angegeben werden. In der Regel ist jedoch `undefined` selten ein gewünschter Wert für nicht belegte

Parameter. Um dieses Problem zu lösen, können seit ECMAScript 2015 für Funktionsparameter auch Standardwerte angegeben werden.

Denken Sie zurück an die Funktion `quadrat(x)`. Diese nimmt ein Argument, multipliziert es mit sich selbst und gibt es zurück:

```
1 function quadrat(x) {  
2   return x * x;  
3 }
```

Würde diese Funktion ohne Parameter aufgerufen, also einfach `quadrat()`, so wäre das Ergebnis `NaN` (not a number). Denn `x` hätte den Wert `undefined` und dies mit sich selbst multipliziert, ergibt keine gültige Zahl.

Um diesen Fehlerfall einfach abzufangen, kann `x` nun mit einem Standardwert, z. B. 0 vorbelegt werden. Dazu muss dieser Wert nur durch ein `=`-Zeichen getrennt neben dem Parameter notiert werden:

```
1 function quadrat(x = 0) {  
2   return x * x;  
3 }
```

Wird nun `quadrat()` aufgerufen, so ist das Ergebnis 0.

8.7.3 Funktionen und der Gültigkeitsbereich von Variablen

Der *Gültigkeitsbereich* (engl. „scope“) einer Variable bestimmt, an welchen Stellen im Programm die Variable gültig, d. h. zugreifbar ist. Unterschieden wird zwischen globalem und lokalem Scope bzw. einer globalen oder lokalen Variable.

Eine globale Variable gilt in der gesamten JavaScript-Datei. Sie wird außerhalb jeder Funktion mit dem `let`- oder `const`- (bzw. `var`-) Schlüsselwort deklariert und gilt somit auch überall im Code – auch innerhalb von Funktionen (außer, sie wird von einer gleichnamigen lokalen Variable überdeckt).

Eine lokale Variable wird innerhalb einer Funktion mit dem `let`- oder `const`- (oder `var`-) Schlüsselwort deklariert und gilt innerhalb der gesamten Funktion bzw. des umgebenden Blocks. Hier dürfen `let`, `const` oder `var` bei der Deklaration auf keinen Fall weggelassen werden, da sonst eine globale Variable erzeugt wird bzw. im strikten Modus ein Fehler erzeugt wird. Lokale Variablen überdecken globale bei Namensgleichheit.

```
1 let scope = "global";  
2  
3 function f() {  
4   let scope = "local";  
5   console.log(scope);  
6 }  
7  
8 f(); // local  
9 console.log(scope); // global
```

Zunächst wird die Variable `scope` angelegt. Da sie außerhalb jeder Funktion deklariert wird, ist sie global. Danach wird die Funktion `f()` angelegt. In ihr wird eine zweite Variable `scope` definiert. Diese hat einen lokalen Gültigkeitsbereich. Da sie gleich wie die globale Variable heißt, überlagert sie in diesem Fall die globale, daher wird beim Aufruf der Funktion `f()` die Zeile `console.log(scope);` auch immer „local“ als Wert

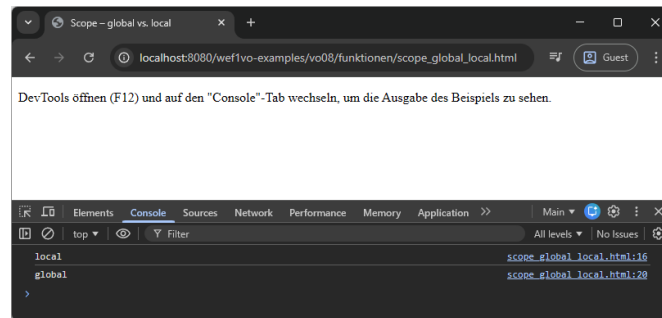


Abbildung 8.1: Die globale Variable wird von der lokalen überdeckt. Die Console zeigt wie erwartet „local“ und „global“ an.

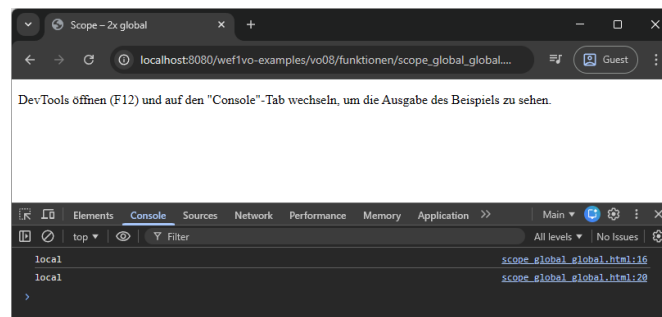


Abbildung 8.2: Die globale Variable wird in der Funktion verändert. Die Console zeigt daher „local“ und „local“ an.

ausgeben. Der zweite Aufruf von `console.log(scope)`; geschieht nun abseits der Funktion. Die dort definierte lokale Variable hat mit dem Ende der Funktion zu existieren aufgehört, daher bleibt nur noch die globale Variable übrig, weshalb der Aufruf auch „global“ zurück gibt. Abbildung 8.1 zeigt das Ergebnis in der Konsole.

Das folgende – leicht abgewandelte Beispiel – demonstriert, was geschieht, wenn die Variable `scope` in der Funktion ohne das `let`-Schlüsselwort verwendet wird. In diesem Fall greift `scope` in der Funktion auf die globale Variable `scope` zu (es wird also nichts überdeckt) und somit wird der Wert der globalen Variable von „global“ auf „local“ geändert. Somit geben sowohl der Funktionsaufruf als auch der Aufruf von `console.log(scope)`; beide Male „local“ aus. Abbildung 8.2 zeigt dies dargestellt in der Konsole.

```
1 let scope = "global";
2
3 function f() {
4   scope = "local";
5   console.log(scope);
6 }
7
8 f(); // local
9 console.log(scope); // local
```


8.7.4 Function Scope vs. Block Scope – var vs. let

Bemerkenswert ist schließlich noch, dass JavaScript vor der Veröffentlichung von ECMAScript 2015 keinen so genannten *Block Scope* besaß. Dieser wirkt sich in anderen Programmiersprachen (wie z. B. C oder Java) so aus, dass Variablen, die in einem Block (ein Block ist alles, das in zwei geschwungenen Klammern steht, wie eine Funktion oder eine `if`-Bedingung) definiert wurden, auch nur dort gelten und außerhalb nicht. In JavaScript traf dies jedoch nicht zu, sodass eine Variable auch außerhalb eines solchen Blocks galt.

Anstatt eines Block Scopes verwendete JavaScript traditionell einen so genannten *Function Scope*. Dies bedeutet, dass Variablen, die innerhalb einer Funktion definiert wurden, in der gesamten Funktion gültig (d. h. deklariert) sind, *egal* in welcher Zeile sie zum ersten Mal angeführt wurde oder anders ausgedrückt, dass Variablen, die in Blöcken innerhalb einer Funktion deklariert wurden, nicht in diesem, sondern im übergeordneten, d. h. in der Funktion gültig waren. Dies wird auch als „Variable hoisting“ (zu Deutsch etwa „Hochziehen von Variablen“) bezeichnet und vom folgenden Beispiel illustriert:

```
1 var scope = "global";
2
3 function f() {
4   var enterBlock = true;
5   console.log(scope); // undefined
6   if (enterBlock) {
7     var scope = "local";
8   }
9   console.log(scope); // local
10 }
11
12 f();
```

Die Variable `enterBlock` wird nur zum Betreten der `if`-Bedingung benötigt. Danach wird zum ersten Mal `console.log(scope)` aufgerufen und `undefined` zurück gegeben. Dies deutet darauf hin, dass nicht die globale Variable ausgegeben wird, sondern die lokale. Diese ist lediglich noch nicht initialisiert worden, aber überdeckt bereits die globale, ist also in der gesamten Funktion gültig. Die Variable selbst wird danach innerhalb der `if`-Bedingung deklariert. Beim zweiten Aufruf von `console.log(scope)` (nachdem `scope` den Wert „local“ zugewiesen bekommen hat), wird auch „local“ ausgegeben. Dies ist nur möglich, da bei der Verwendung von `var` zur Variablendeklaration kein Block Scope existiert und somit `scope` auch nach Ende der `if`-Bedingung noch existiert.

Mit der Einführung des Schlüsselworts `let` unter ECMAScript 2015 ist es nun auch möglich, das eher gewohnte Block Scope-Verhalten zu erzeugen. Jede, mit `let` deklarierte Variable, ist nur in dem Block gültig, in dem sie angelegt wurde. Ein leicht modifiziertes Beispiel demonstriert dies:

```
1 let scope = "global";
2
3 function f() {
4   let enterBlock = true;
5   console.log(scope); // global
6   if (enterBlock) {
7     let scope = "local";
8     console.log(scope); // local
```

```
9   }  
10  console.log(scope); // global  
11 }  
12  
13 f();
```

Zunächst wird eine globale Variable `scope` angelegt. Hier macht es keinen Unterschied, ob `let` oder `var` verwendet wird. Ebenso bei der Definition der Variable `enterBlock`. Da diese auf Funktionsebene angelegt wird, gilt sie – egal ob mit `let` oder `var` – in der gesamten Funktion (da dies der umgebende Block ist). Innerhalb der `if`-Bedingung bewirkt `let`, dass die lokale Variable `scope` nur in derselben gültig ist. Somit überlagert sie nur hier die globale Variable, beim ersten Aufruf von `console.log(scope)` jedoch nicht (weswegen dort auch „global“ und nicht „undefined“ ausgegeben wird).

Das doch ungewohnte Verhalten eines Function Scopes ist der Grund, weshalb eine Variablendeklaration mittels `let` auf jeden Fall bevorzugt werden sollte.

8.7.5 Funktionen: Ein Beispiel

Das folgende Beispiel für eine Funktion prüft ab, ob es sich bei einer Zahl um eine Primzahl handelt. Zunächst wird die Funktion definiert und danach mit einer Zahl aufgerufen. Das Ergebnis wird in der Konsole angezeigt.

```
1 function isPrime(number) {  
2   let isPrimeNumber = true;  
3   let i = 2;  
4   while (i * i <= number && isPrimeNumber) {  
5     if (number % i === 0) {  
6       isPrimeNumber = false;  
7     }  
8     i++;  
9   }  
10  return isPrimeNumber;  
11 }  
12  
13 const x = 5;  
14 console.log("%d ist %s Primzahl", x, isPrime(x) ? "eine" : "keine");
```

Der Funktion wird eine Zahl `number` übergeben. Die Funktion merkt sich mit der booleschen Variable `isPrimeNumber` ob die Zahl eine Primzahl ist. Zunächst wird immer davon ausgegangen. Danach wird die zu überprüfende Zahl beginnend bei 2 durch jede Zahl dividiert, die kleiner als $\sqrt{\text{number}}$ ist. Genauer gesagt wird der Modulo-Operator verwendet, denn wenn eine Division keinen Rest erzeugt (Modulo ergibt 0), dann wurde eine Zahl gefunden, durch die `number` teilbar ist. Somit kann es sich um keine Primzahl mehr handeln. Schließlich gibt die Funktion `true` oder `false` zurück, je nachdem, ob es sich um eine Primzahl handelt oder nicht.

Danach wird die Funktion aufgerufen – in diesem Fall mit der Variable `x`, die den Wert 5 hat. In der letzten Zeile wird der Output in der Konsole generiert, dabei wird mit Hilfe des konditionalen Operators entweder „eine“ oder „keine“ eingesetzt, je nachdem, ob `isPrime(x)` `true` oder `false` zurück liefert.

8.7.6 Anonyme Funktionen und Arrow Functions

Im Verlauf dieses Abschnitts wurden Funktionen immer mit einem Namen definiert (`function name (...) { }`). Da Funktionen in JavaScript jedoch wie Werte behandelt werden, können sie auch ohne Namen erstellt und direkt in Variablen gespeichert werden. Dies nennt man *anonyme Funktionen*.

Seit ECMAScript 2015 gibt es dafür eine besonders kompakte Schreibweise: die *Arrow Functions* (Pfeilfunktionen). Sie sind das JavaScript-Äquivalent zu den Lambda-Ausdrücken in Java, verwenden jedoch einen „fetten Pfeil“ (`=>`) statt des Bindestrichs (`->`).

Klassische anonyme Funktionen werden mit dem Schlüsselwort `function()` erstellt, es folgt jedoch kein Funktionsname. Dieser ist nicht nötig, wenn die Funktion einer Variable zugewiesen wird, denn über deren Namen kann sie aufgerufen werden:

```
1 const calcAsAnonFunction = function(x) {  
2   return x * x;  
3 };  
4  
5 console.log("Anonyme Funktion:", calcAsAnonFunction(4));
```

Arrow Functions verzichten auf das Schlüsselwort `function()`. Es wird nur eine runde Klammer gesetzt, die Parameter enthalten kann (nicht muss). Danach folgt ein Pfeil `=>` und dann die geschwungenen Klammern mit dem Code.

```
1 const calcAsArrowFunction = (x) => {  
2   return x * x;  
3 };  
4  
5 console.log("Arrow Funktion:", calcAsArrowFunction(4));
```

Hat eine Arrow Function genau einen Parameter, können die runden Klammern um diesen weggelassen werden. Besteht der Funktionskörper zudem aus nur einer einzigen Anweisung, können auch die geschwungenen Klammern und das `return` entfallen. Dies macht die Syntax extrem kompakt:

```
1 const calcAsShortArrowFunction = x => x * x;  
2  
3 console.log("Kurze Arrow-Funktion:", calcAsShortArrowFunction(4));
```

Wann wird welche Syntax verwendet?

Arrow Functions haben sich in der modernen JavaScript-Entwicklung für kurze, prägnante Operationen durchgesetzt.

Vorteile von Arrow Functions:

- **Kürze:** Der Code wird deutlich lesbarer, besonders bei Einzeilern.
- **Kontext (`this`):** Arrow Functions haben keinen eigenen `this`-Kontext, sondern erben ihn aus der Umgebung. Dies ist ein entscheidender Vorteil bei der Programmierung von Klassen oder Event-Handleern (mehr dazu in den Vorlesungen 9 und 10).

Nachteile/Einschränkungen:

- Sie besitzen kein eigenes **arguments**-Objekt (wie in Abschnitt 8.7.2 erwähnt).
- Sie können nicht als Konstruktor für Objekte verwendet werden.

In der Praxis werden Arrow Functions vor allem dann verwendet, wenn eine Funktion als Argument an eine andere Funktion übergeben wird (sogenannte *Callbacks*), etwa bei Event-Handlern („Klick auf Button“) oder bei der Verarbeitung von Arrays. Mehr zu Events in Vorlesung 10.

8.7.7 Vordefinierte Funktionen

JavaScript (bzw. genauer ECMAScript) bietet eine Reihe von vordefinierten Funktionen. Die folgende Aufzählung ist nur ein Ausschnitt, die vollständige Liste kann unter [3, Abschnitt 19.2] oder beim MDN⁷ nachgeschlagen werden.

- **isFinite(number)**: Überprüft, ob es sich bei **number** um eine Zahl handelt oder nicht. Gibt **true** oder **false** zurück.
- **isNaN(value)**: Überprüft, ob **value** keine Zahl (not a number, NaN) ist. Gibt **true** oder **false** zurück.
- **parseFloat(string)**: Wandelt einen String, der eine Gleitkommazahl enthält (z. B. aus einem Eingabeformular) in eine Zahl um und gibt diese zurück.
- **parseInt(string, radix)**: Wandelt einen String, der eine Ganzzahl enthält (z. B. aus einem Formular) in eine Zahl um und gibt diese zurück. Mit dem zweiten Parameter wird der Radix (die Basis des Zahlensystems angegeben; Standard ist 10).
- **encodeURIComponent(uri)**: Kodiert einen vollständigen Uniform Resource Identifier (URI), indem Sonderzeichen durch UTF-8-Escape-Sequenzen ersetzt werden (z. B. wird ein Leerzeichen zu %20). Steuerzeichen wie /, : oder ? bleiben dabei erhalten.

8.8 Weiterführende Literatur

Das derzeit wohl umfangreichste und aktuellste Werk ist [1], das bereits ECMAScript 2015 und neuer abdeckt und auch sonst alle Details zur Sprache auf über 1.200 Seiten enthält.

Ebenso empfehlenswert ist [6], das etwas aktueller, mit ca. 500 Seiten jedoch etwas knapper ausgeführt ist.

Das langjährige Standardwerk zu JavaScript von David Flanagan [4] hat 2020 eine neue Auflage bekommen und kann somit auch noch als (englischsprachige) Referenz angesehen werden. Die vorangegangenen Auflagen sind aufgrund ihres Alters nicht mehr empfehlenswert.

Schließlich findet sich in [7] auch eine Einführung in die Thematik, die zwar oberflächlich bleibt, aber zum Erlernen der Sprache vielen auch ausreichen dürfte.

⁷https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects#function_properties

Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 8-27).
- [2] Mathias Bynens. *Valid JavaScript variable names in ES2015*. 4. März 2015. URL: <https://mathiasbynens.be/notes/javascript-identifiers-es6> (besucht am 12.12.2025) (siehe S. 8-9).
- [3] European Computer Manufacturers Association. *ECMAScript® 2025 Language Specification*. ECMA-262. Version 16.0. Juni 2025. URL: <https://ecma-international.org/publications-and-standards/standards/ecma-262/> (siehe S. 8-2, 8-3, 8-27).
- [4] David Flanagan. *JavaScript. The Definitive Guide*. 7. Aufl. Sebastopol, USA: O'Reilly, 9. Juni 2020 (siehe S. 8-27).
- [5] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754™-2019. 13. Juni 2019. URL: <https://ieeexplore.ieee.org/document/8766229> (siehe S. 8-4).
- [6] Thomas Theis. *Einstieg in JavaScript*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Juli 2024 (siehe S. 8-27).
- [7] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 8-27).
- [8] Juriy Zaytsev, Leon Arnott und Denis Pushkarev. *ECMAScript compatibility table*. 11. Nov. 2025. URL: <https://compat-table.github.io/compat-table/> (besucht am 12.12.2025) (siehe S. 8-3, 8-10).